

# SPRING-REST-HANDSON-1 Exercises

## Hands on 1

### Create a Spring Web Project using Maven

Follow steps below to create a project:

1. Go to <https://start.spring.io/>
2. Change Group as “com.cognizant”
3. Change Artifact Id as “spring-learn”
4. Select Spring Boot DevTools and Spring Web
5. Create and download the project as zip
6. Extract the zip in root folder to Eclipse Workspace
7. Build the project using ‘mvn clean package -Dhttp.proxyHost=proxy.cognizant.com -Dhttp.proxyPort=6050 -Dhttps.proxyHost=proxy.cognizant.com -Dhttps.proxyPort=6050 -Dhttp.proxyUser=123456’ command in command line
8. Import the project in Eclipse "File > Import > Maven > Existing Maven Projects > Click Browse and select extracted folder > Finish"
9. Include logs to verify if main() method of SpringLearnApplication.
10. Run the SpringLearnApplication class.

Output:

```
:: Spring Boot :: (v4.1.0)
2026-07-02T23:23:52.632+05:30 INFO 12168 --- [spring-learn] [main] c.c.springlearn.SpringLearnApplication : Starting SpringLearnApplication using Java
17.0.12 with PID 12168 (D:\PROJECTS\SPRING PROJECTS\spring-learn\target\classes started by user in D:\PROJECTS\SPRING PROJECTS\spring-learn)
2026-07-02T23:23:52.637+05:30 INFO 12168 --- [spring-learn] [main] c.c.springlearn.SpringLearnApplication : No active profile set, falling back to 1 default
profile: "default"
2026-07-02T23:23:53.866+05:30 INFO 12168 --- [spring-learn] [main] c.c.springlearn.SpringLearnApplication : Started SpringLearnApplication in 1.746 seconds
(process running for 2.229)
```

```
Starting SpringLearnApplication using Java 17.0.12 with PID 12168 (D:\PROJECTS\SPRING PROJECTS\spring-learn\target\classes started by user in D:\PROJECTS\
No active profile set, falling back to 1 default profile: "default"
Started SpringLearnApplication in 1.746 seconds (process running for 2.229)
```

The Downloaded File has spring-learn but not springlearn package. So changed the codes according to feasibility.

SME to walk through the following aspects related to the project created:

1. **src/main/java** - Folder with application code
2. **src/main/resources** - Folder for application configuration
3. **src/test/java** - Folder with code for testing the application
4. **SpringLearnApplication.java** - Walkthrough the main() method.
5. Purpose of **@SpringBootApplication** annotation

It works like power button of a computer. Where it automatically starts all the processing's once the computer is opened. Likewise this acts the same for the given project.

6. **pom.xml**
  1. Walkthrough all the configuration defined in XML file
  2. Open 'Dependency Hierarchy' and show the dependency tree.

The screenshot displays the IDE's 'Dependency Hierarchy' and 'Resolved Dependencies' views. The 'Dependency Hierarchy' view on the left shows a tree structure of dependencies, including spring-boot-starter, spring-boot-starter-logging, logback-classic, logback-core, slf4j-api, log4j-to-slf4j, log4j-api, jul-to-slf4j, spring-boot-autoconfigure, spring-boot, spring-core, spring-context, spring-aop, spring-beans, and spring-expression. The 'Resolved Dependencies' view on the right lists the resolved versions of these dependencies, such as opentest4j:1.3.0, slf4j-api:2.0.18, snakeyaml:2.6, spring-aop:7.0.8, spring-beans:7.0.8, spring-boot:4.1.0, spring-boot-autoconfigure:4.1.0, spring-boot-starter:4.1.0, spring-boot-starter-logging:4.1.0, spring-boot-starter-test:4.1.0, spring-boot-test:4.1.0, spring-boot-test-autoconfigure:4.1.0, spring-context:7.0.8, spring-core:7.0.8, spring-expression:7.0.8, spring-test:7.0.8, and xmlunit-core:2.11.0. Below these views is a terminal window showing the output of running the application, which displays the Spring Boot logo and version information.

```
Dependency Hierarchy [test]
Filter:
Dependency Hierarchy
Resolved Dependencies
Overview Dependencies Dependency Hierarchy Effective POM pom.xml
Console Terminal
terminated> SpringLearnApplication [Java Application] C:\Program Files\Java\jdk-17\bin\javaw.exe (2 Jul 2026, 11:23:51 pm - 11:23:53 pm elapsed 0:00:02) [pid: 12168]
:: Spring Boot :: (v4.1.0)
2026-07-02T23:23:52.632+05:30 INFO 12168 --- [spring-learn] [main] c.c.springlearn.SpringLearnApplication : Starting SpringLearnApplication usi
2026-07-02T23:23:52.637+05:30 INFO 12168 --- [spring-learn] [main] c.c.springlearn.SpringLearnApplication : No active profile set, falling back
```

## Spring Core – Load SimpleDateFormat from Spring Configuration XML

**SimpleDateFormat** with the pattern 'dd/MM/yyyy' is created in multiple places of an application. To avoid creation of SimpleDateFormat in multiple places, define a bean in Spring XML Configuration file and retrieve the date.

Follow steps below to implement:

- Create spring configuration file **date-format.xml** in **src/main/resources** folder of 'spring-learn' project
- Open <https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html#beans-factory-metadata>
- Copy the XML defined in the section of previous step URL and paste it into **date-format.xml**
- Define bean tag in the XML with for date format. Refer code below.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dateFormat" class="java.text.SimpleDateFormat">
        <constructor-arg value="dd/MM/yyyy" />
    </bean>

</beans>
```

- Create new method **displayDate()** in **SpringLearnApplication.java**
- In displayDate() method create the **ApplicationContext**. Refer code below:

```
ApplicationContext context = new ClassPathXmlApplicationContext("date-
format.xml");
```

- Get the `dateFormat` using `getBean()` method. Refer code below.

```
SimpleDateFormat format = context.getBean("dateFormat",  
SimpleDateFormat.class);
```

- Using the format variable try to parse string '`31/12/2018`' to `Date` class and display the result using `System.out.println`.
- Run the application as 'Java Application' and check the result in console log output.

```
2026-07-03T00:02:03.438+05:30 INFO 6112 --- [spring-learn  
2026-07-03T00:02:03.443+05:30 INFO 6112 --- [spring-learn  
2026-07-03T00:02:03.447+05:30 INFO 6112 --- [spring-learn  
Mon Dec 31 00:00:00 IST 2018  
|
```

```
2026-07-03T00:01:14.026+05:30 INFO 6112 --- [spring-learn  
2026-07-03T00:01:14.557+05:30 INFO 6112 --- [spring-learn  
2026-07-03T00:01:14.610+05:30 INFO 6112 --- [spring-learn  
2026-07-03T00:01:14.627+05:30 INFO 6112 --- [spring-learn  
Wed Dec 27 00:00:00 IST 2006
```

## Troubleshooting Tips

If the tomcat port has a conflict and the server is not starting include the below property in `application.properties` file in `src/main/resources` folder.

## Spring Core - Incorporate Logging

Incorporate logging in the Spring Boot project created in previous hands on. Refer steps below:

- Create **application.properties** if not yet created in **src/main/resources** folder
- Add below lines in **application.properties**

```
logging.level.org.springframework=info
logging.level.com.cognizant.springlearn=debug
logging.pattern.console=%d{yyMMdd} | %d{HH:mm:ss.SSS} | %-20.20thread | %5p | %-25.25logger{25} | %25M | %m%n
```

- In **SpringLearnApplication.java** include the following imports:

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
```

- Include the below static variable in **SpringLearnApplication.java**:

```
private static final Logger LOGGER =
    LoggerFactory.getLogger(SpringLearnApplication.class);
```

- Include info log on start and end of method. Debug log for displaying the date (refer code below)

```
public void displayDate() {
    LOGGER.info("START");
    //..
    LOGGER.debug(date);
    //..
    LOGGER.info("END");
}
```

**IMPORTANT NOTE:** Going forward all methods should incorporate logging as specified above. **Never** use `System.out.println()`.

```

logStarting|Starting SpringLearnApplication using Java 17.0.12 with PID 6112 (C:\Users\user\Downloads\Sai Abhiram Yadlapalli\STUDIES\SELF-PLACED\Cognizant\JavaFSE\Spring REST using Spring Boot\
logStarting|Running with Spring Boot v3.5.16, Spring v6.2.19
StartupProfileInfo|No active profile set, falling back to 1 default profile: "default"
initialize|Tomcat initialized with port 8080 (http)
log|Starting service [Tomcat]
log|Starting Servlet engine: [Apache Tomcat/10.1.55]
log|Initializing Spring embedded WebApplicationContext
log|Root WebApplicationContext: initialization completed in 187 ms
startServer|LiveReload server is running on port 35729
start|Tomcat started on port 8080 (http) with context path '/'
logStarted|Started SpringLearnApplication in 0.275 seconds (process running for 605.796)
ApplicationEvent|Condition evaluation unchanged
displayDate|START
displayDate|Date : Mon Dec 31 00:00:00 IST 2018
displayDate|END
main|Application Ended

```

#### Hands on 4

## Spring Core – Load Country from Spring Configuration XML

An airlines website is going to support booking on four countries. There will be a drop down on the home page of this website to select the respective country. It is also important to store the two-character ISO code of each country.

| Code | Name          |
|------|---------------|
| US   | United States |
| DE   | Germany       |
| IN   | India         |
| JP   | Japan         |

Above data has to be stored in spring configuration file. Write a program to read this configuration file and display the details.

#### Steps to implement

- Pick any one of your choice country to configure in Spring XML configuration named **country.xml**.
- Create a bean tag in spring configuration for country and set the property and values

```

<bean id="country" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN" />
    <property name="name" value="India" />
</bean>

```

- Create **Country** class with following aspects:

- Instance variables for **code** and **name**
- Implement empty parameter constructor with inclusion of debug log within the constructor with log message as “**Inside Country Constructor.**”
- Generate getters and setters with inclusion of debug with relevant message within each setter and getter method.
- Generate **toString()** method
- Create a method **displayCountry()** in **SpringLearnApplication.java**, which will read the country bean from spring configuration file and display the country details. **ClassPathXmlApplicationContext**, **ApplicationContext** and **context.getBean(“beanId”, Country.class)**. Refer sample code for **displayCountry()** method below.

```
ApplicationContext context = new ClassPathXmlApplicationContext("country.xml");
Country country = (Country) context.getBean("country", Country.class);
LOGGER.debug("Country : {}", country.toString());
```

- Invoke **displayCountry()** method in **main()** method of **SpringLearnApplication.java**.
- Execute **main()** method and check the logs to find out which constructors and methods were invoked.

SME to provide more detailing about the following aspects:

- bean tag, id attribute, class attribute, property tag, name attribute, value attribute
- **ApplicationContext**, **ClassPathXmlApplicationContext**
- What exactly happens when **context.getBean()** is invoked

```
260703|00:20:57.419|restartedMain|INFO|.s.SpringLearnApplication|onApplicationEvent|condition evaluation unchanged
260703|00:20:57.419|restartedMain|INFO|.s.SpringLearnApplication|displayDate|START
260703|00:20:57.428|restartedMain|DEBUG|.s.SpringLearnApplication|displayDate|Date : Mon Dec 31 00:00:00 IST 2018
260703|00:20:57.429|restartedMain|INFO|.s.SpringLearnApplication|displayDate|END
260703|00:20:57.429|restartedMain|INFO|.s.SpringLearnApplication|displayCountry|START
260703|00:20:57.437|restartedMain|DEBUG|.c.c.spring_learn.Country|<init>|Inside Country Constructor.
260703|00:20:57.438|restartedMain|DEBUG|.c.c.spring_learn.Country|setCode|Inside setCode()
260703|00:20:57.438|restartedMain|DEBUG|.c.c.spring_learn.Country|setName|Inside setName()
260703|00:20:57.439|restartedMain|DEBUG|.s.SpringLearnApplication|displayCountry|Country : Country [code=IN, name=India]
260703|00:20:57.440|restartedMain|INFO|.s.SpringLearnApplication|displayCountry|END
260703|00:20:57.440|restartedMain|INFO|.s.SpringLearnApplication|main|Application Ended
```

application.properties :

```
spring.application.name=spring-learn
logging.level.org.springframework=info
logging.level.com.cognizant.spring_learn=debug
logging.pattern.console=%d{yyMMdd} | %d{HH:mm:ss.SSS} | %-20.20thread | %5p | %-
25.25logger{25} | %25M | %m%n
```

data-format.xml :

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="dateFormat" class="java.text.SimpleDateFormat">
        <constructor-arg value="dd/MM/yyyy"/>
    </bean>

</beans>
```

country.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="country" class="com.cognizant.spring_learn.Country">
        <property name="code" value="IN"/>
        <property name="name" value="India"/>
    </bean>

</beans>
```

Country.java :

```
package com.cognizant.spring_learn;

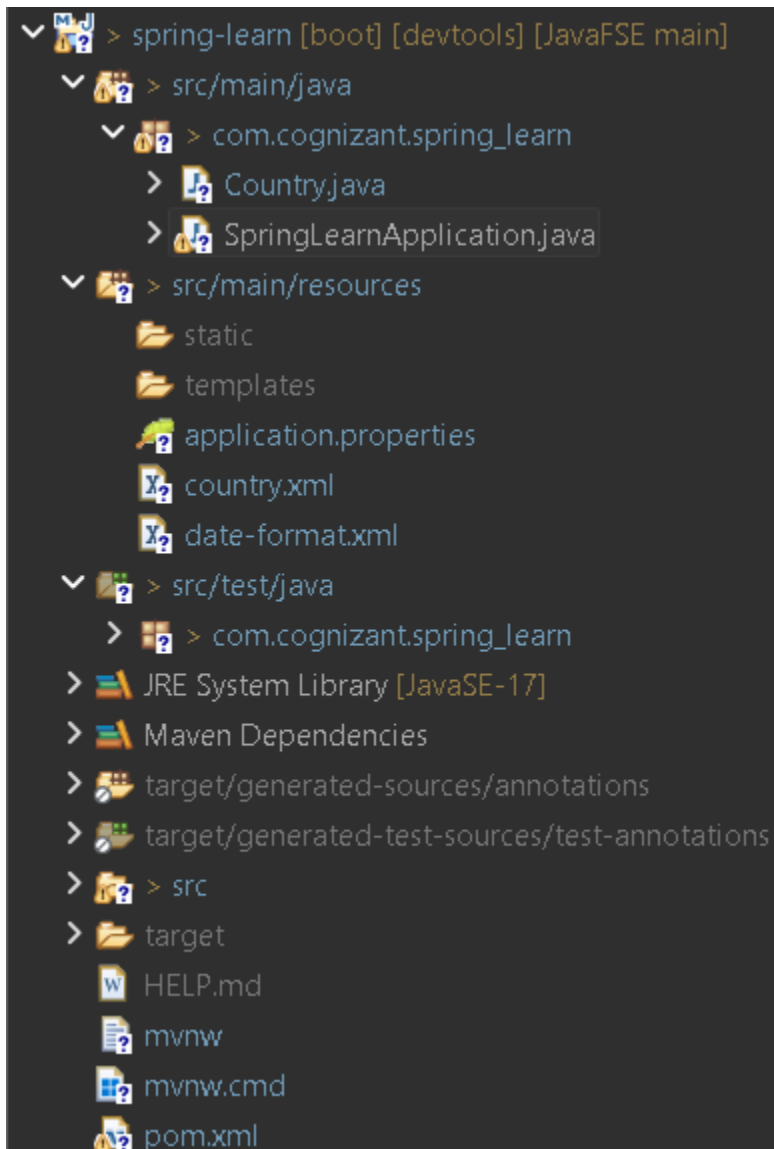
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class Country {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(Country.class);
    private String code;
    private String name;
    public Country() {
        LOGGER.debug("Inside Country Constructor.");
    }
    public String getCode() {
        LOGGER.debug("Inside getCode()");
        return code;
    }
    public void setCode(String code) {
        LOGGER.debug("Inside setCode()");
        this.code = code;
    }
    public String getName() {
        LOGGER.debug("Inside getName()");
        return name;
    }
    public void setName(String name) {
        LOGGER.debug("Inside setName()");
        this.name = name;
    }
    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "];"
    }
}
```

SpringLearnApplication.java :

```
package com.cognizant.spring_learn;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.text.SimpleDateFormat;
import java.util.Date;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
@SpringBootApplication
public class SpringLearnApplication {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);
    public static void main(String[] args) throws Exception {
        LOGGER.info("Application Started");
        SpringApplication.run(SpringLearnApplication.class, args);
        displayDate();
        displayCountry();
        LOGGER.info("Application Ended");
    }
    public static void displayDate() throws Exception {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("date-format.xml");
        SimpleDateFormat format =
            context.getBean("dateFormat", SimpleDateFormat.class);
        Date date = format.parse("31/12/2018");
        LOGGER.debug("Date : {}", date);
        LOGGER.info("END");
    }
    public static void displayCountry() {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        LOGGER.debug("Country : {}", country);
        LOGGER.info("END");
    }
}
```

Total Project :



# SPRING-REST-HANDSON-2 Exercises

## Hello World RESTful Web Service

Write a REST service in the spring learn application created earlier, that returns the text "Hello World!!" using Spring Web Framework. Refer details below:

**Method:** GET

**URL:** /hello

**Controller:** com.cognizant.spring-learn.controller.HelloController

**Method Signature:** public String sayHello()

**Method Implementation:** return hard coded string "Hello World!!"

**Sample Request:** http://localhost:8083/hello

**Sample Response:** Hello World!!

**IMPORTANT NOTE:** Don't forget to include start and end log in the sayHello() method.

Try the URL <http://localhost:8083/hello> in both chrome browser and postman.

SME to explain the following aspects:

- In network tab of developer tools show the HTTP header details received
- In postman click on "Headers" tab to view the HTTP header details received

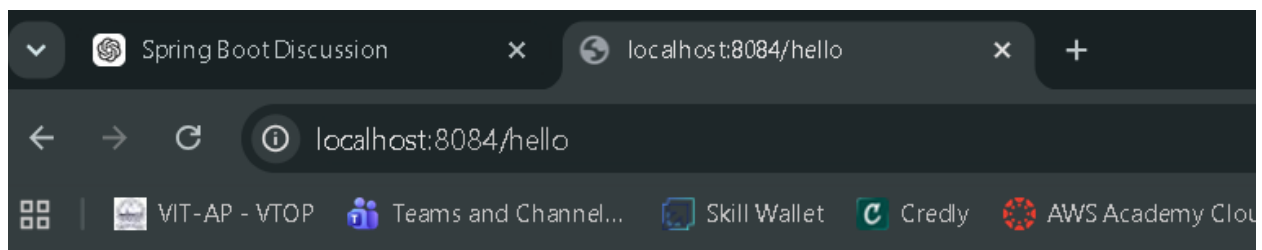
## HelloController.java :

```
package com.cognizant.spring_learn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(HelloController.class);
    @GetMapping("/hello")
    public String sayHello() {
        LOGGER.info("START");
        String message = "Hello World!!";
        LOGGER.info("END");
        return message;
    }
}
```

PORT=8083 is busy. So used 8084 port number.



Hello World!!

```
00:45:47.640 [main] INFO com.cognizant.spring_learn.SpringLearnApplication --  
Application Started  
00:45:47.863 [restartedMain] INFO com.cognizant.spring_learn.SpringLearnApplication -  
- Application Started  
  
      .\ /_____. _ _ _ _ _ \\\_\\\ _\\_\\_\\_\\  
( ( ) \\___| | | | | | | | | | V ___| \\ \\ \\ \\ \\ \\  
\\V ____| | | | | | | | | | ( | | ) ) ) )  
' |___| | | | | | | | | | \\ __, | // // //  
=====|_|=====|___/=//////  
  
:: Spring Boot ::                (v3.5.16)  
  
260703|00:45:48.416|restartedMain          | INFO|.s.SpringLearnApplication|  
logStarting|Starting SpringLearnApplication using Java 17.0.12 with PID 21700  
(C:\Users\user\Downloads\Sai AbhiRam Yadlapalli\STUDIES\SELF-  
PLACED\Cognizant\JavaFSE\Spring REST using Spring Boot\spring-learn\target\classes
```

```
started by user in C:\Users\user\Downloads\Sai AbhiRam Yadlapalli\STUDIES\SELF-
PLACED\Cognizant\JavaFSE\Spring REST using Spring Boot\spring-learn)
260703|00:45:48.419|restartedMain      |DEBUG|.s.SpringLearnApplication|
logStarting|Running with Spring Boot v3.5.16, Spring v6.2.19
260703|00:45:48.421|restartedMain      | INFO|.s.SpringLearnApplication|
logStartupProfileInfo|No active profile set, falling back to 1 default profile:
"default"
260703|00:45:48.500|restartedMain      | INFO|ertyDefaultsPostProcessor|
log|Devtools property defaults active! Set 'spring.devtools.add-properties' to
'false' to disable
260703|00:45:48.501|restartedMain      | INFO|ertyDefaultsPostProcessor|
log|For additional web related logging consider setting the 'logging.level.web'
property to 'DEBUG'
260703|00:45:49.728|restartedMain      | INFO|s.b.w.e.t.TomcatWebServer|
initialize|Tomcat initialized with port 8083 (http)
260703|00:45:49.748|restartedMain      | INFO|o.a.c.c.StandardService  |
log|Starting service [Tomcat]
260703|00:45:49.749|restartedMain      | INFO|o.a.c.core.StandardEngine|
log|Starting Servlet engine: [Apache Tomcat/10.1.55]
260703|00:45:49.812|restartedMain      | INFO|o.a.c.c.C.[.].[/]      |
log|Initializing Spring embedded WebApplicationContext
260703|00:45:49.817|restartedMain      |
INFO|bServerApplicationContext|prepareWebApplicationContext|Root
WebApplicationContext: initialization completed in 1311 ms
260703|00:45:50.381|restartedMain      | WARN|.OptionalLiveReloadServer|
startServer|Unable to start LiveReload server
260703|00:45:50.406|restartedMain      | WARN|bServerApplicationContext|
refresh|Exception encountered during context initialization - cancelling refresh
attempt: org.springframework.context.ApplicationContextException: Failed to start
bean 'webServerStartStop'
260703|00:45:50.431|restartedMain      | INFO|ionEvaluationReportLogger|
logMessage|
```

Error starting ApplicationContext. To display the condition evaluation report re-run your application with 'debug' enabled.

```
260703|00:45:50.461|restartedMain      |ERROR|ngFailureAnalysisReporter|
report|
```

```
*****
APPLICATION FAILED TO START
*****
```

#### Description:

Web server failed to start. Port 8083 was already in use.

#### Action:

Identify and stop the process that's listening on port 8083 or configure this application to listen on another port.

## REST - Country Web Service

Write a REST service that returns India country details in the earlier created spring learn application.

**URL:** /country

**Controller:** com.cognizant.spring-learn.controller.CountryController

**Method Annotation:** @RequestMapping

**Method Name:** getCountryIndia()

**Method Implementation:** Load India bean from spring xml configuration and return

**Sample Request:** http://localhost:8083/country

**Sample Response:**

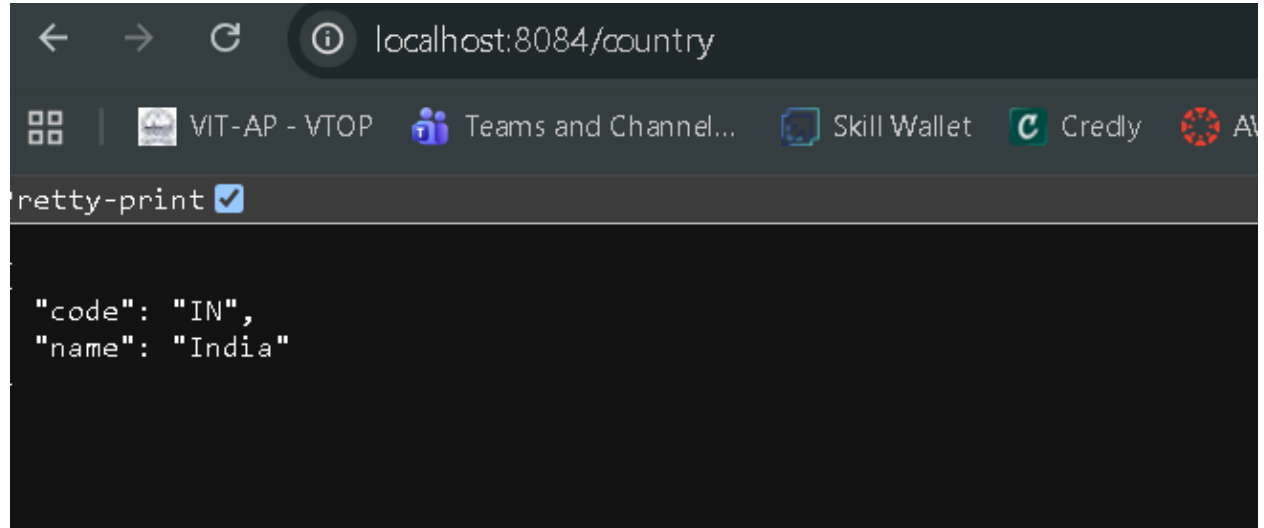
```
{
  "code": "IN",
  "name": "India"
}
```

SME to explain the following aspects:

- What happens in the controller method?
- How the bean is converted into JSON response?
- In network tab of developer tools show the HTTP header details received
- In postman click on "Headers" tab to view the HTTP header details received

## CountryController.java

```
package com.cognizant.spring_learn.controller;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.spring_learn.Country;
@RestController
public class CountryController {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);
    @RequestMapping("/country")
    public Country getCountryIndia() {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        LOGGER.info("END");
        return country;
    }
}
```



## REST - Get all countries

Write a REST service that returns all the countries.

**Controller:** com.cognizant.spring-learn.controller.CountryController

**Method Annotation:** @GetMapping("/countries")

**Method Name:** getAllCountries()

**Method Implementation:** Load country list from country.xml and return

**Sample Request:** http://localhost:8083/countries

**Sample Response:**

```
[
  { "code": "IN", "name": "India"},
  { "code": "US", "name": "United States"},
  { "code": "JP", "name": "Japan"},
  { "code": "DE", "name": "Germany"}
]
```

SpringLearnApplications.java :

```
package com.cognizant.spring_learn;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.text.SimpleDateFormat;
import java.util.Date;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
@SpringBootApplication
public class SpringLearnApplication {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);
    public static void main(String[] args) throws Exception {
        LOGGER.info("Application Started");
        SpringApplication.run(SpringLearnApplication.class, args);
        displayDate();
        displayCountry();
        LOGGER.info("Application Ended");
    }
    public static void displayDate() throws Exception {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("date-format.xml");
        SimpleDateFormat format =
```

```

        context.getBean("dateFormat", SimpleDateFormat.class);
        Date date = format.parse("31/12/2018");
        LOGGER.debug("Date : {}", date);
        LOGGER.info("END");
    }
    public static void displayCountry() {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        LOGGER.debug("Country : {}", country);
        LOGGER.info("END");
    }
}

```

CountryController.java :

```

package com.cognizant.spring_learn.controller;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.spring_learn.Country;
import java.util.List;
import org.springframework.web.bind.annotation.GetMapping;
@RestController
public class CountryController {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);
    @RequestMapping("/country")
    public Country getCountryIndia() {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        LOGGER.info("END");
        return country;
    }
    @GetMapping("/countries")
    public List<Country> getAllCountries() {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        List<Country> countryList =
            context.getBean("countryList", List.class);
        LOGGER.info("END");
        return countryList;
    }
}

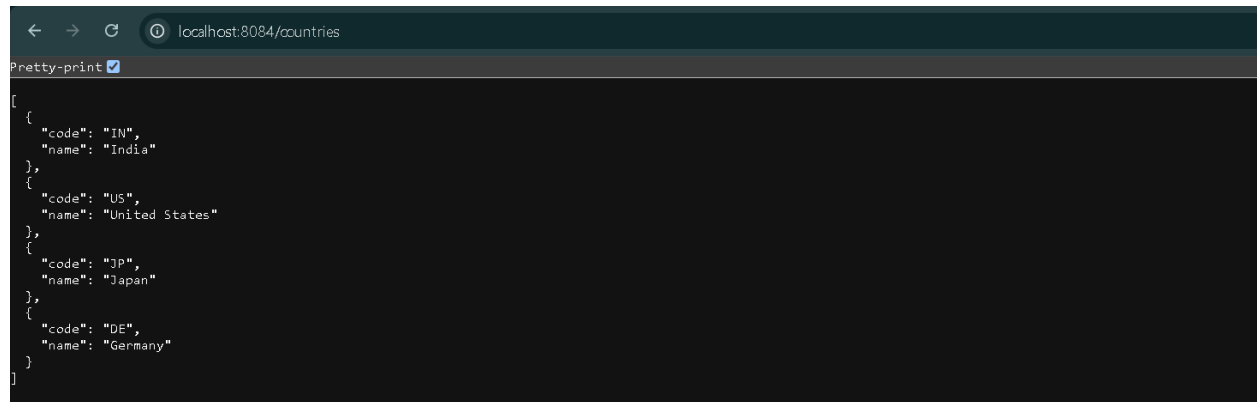
```

country.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="country" class="com.cognizant.spring_learn.Country">
    <property name="code" value="IN"/>
    <property name="name" value="India"/>
  </bean>
  <bean id="us" class="com.cognizant.spring_learn.Country">
    <property name="code" value="US"/>
    <property name="name" value="United States"/>
  </bean>
  <bean id="jp" class="com.cognizant.spring_learn.Country">
    <property name="code" value="JP"/>
    <property name="name" value="Japan"/>
  </bean>
  <bean id="de" class="com.cognizant.spring_learn.Country">
    <property name="code" value="DE"/>
    <property name="name" value="Germany"/>
  </bean>
  <bean id="countryList" class="java.util.ArrayList">
    <constructor-arg>
      <list>
        <ref bean="country"/>
        <ref bean="us"/>
        <ref bean="jp"/>
        <ref bean="de"/>
      </list>
    </constructor-arg>
  </bean>
</beans>
```

Output:



A screenshot of a web browser window with the address bar showing 'localhost:8084/countries'. Below the address bar, there is a 'pretty-print' checkbox which is checked. The main content area displays a JSON array of four objects, each representing a country with its 'code' and 'name'.

```
[
  {
    "code": "IN",
    "name": "India"
  },
  {
    "code": "US",
    "name": "United States"
  },
  {
    "code": "JP",
    "name": "Japan"
  },
  {
    "code": "DE",
    "name": "Germany"
  }
]
```



A screenshot of a web browser window, similar to the one above, showing 'localhost:8084/countries'. The 'pretty-print' checkbox is checked. The JSON response is now formatted with proper indentation and line breaks, making it easier to read.

```
[
  {
    "code": "IN",
    "name": "India"
  },
  {
    "code": "US",
    "name": "United States"
  },
  {
    "code": "JP",
    "name": "Japan"
  },
  {
    "code": "DE",
    "name": "Germany"
  }
]
```

## REST - Get country based on country code

Write a REST service that returns a specific country based on country code. The country code should be case insensitive.

**Controller:** com.cognizant.spring-learn.controller.CountryController

**Method Annotation:** @GetMapping("/countries/{code}")

**Method Name:** getCountry(String code)

**Method Implementation:** Invoke countryCodeService.getCountry(code)

**Service Method:** com.cognizant.spring-learn.service.CountryService.getCountry(String code)

### Service Method Implementation:

- Get the country code using @PathVariable
- Get country list from country.xml
- Iterate through the country list
- Make a case insensitive matching of country code and return the country.
- Lambda expression can also be used instead of iterating the country list

**Sample Request:** http://localhost:8083/country/in

### Sample Response:

```
{
  "code": "IN",
  "name": "India"
}
```

## Country Controller.java

```
package com.cognizant.spring_learn.controller;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.spring_learn.Country;
import java.util.List;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.PathVariable;
import com.cognizant.spring_learn.service.CountryService;
@RestController
public class CountryController {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(CountryController.class);
    @Autowired
    private CountryService countryService;
    @RequestMapping("/country")
    public Country getCountryIndia() {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        LOGGER.info("END");
        return country;
    }
    @GetMapping("/countries")
    public List<Country> getAllCountries() {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        List<Country> countryList =
            context.getBean("countryList", List.class);
        LOGGER.info("END");
        return countryList;
    }
    @GetMapping("/countries/{code}")
    public Country getCountry(@PathVariable String code) {
        LOGGER.info("START");
        Country country = countryService.getCountry(code);
        LOGGER.info("END");
        return country;
    }
}
```

country.xml :

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    https://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="country" class="com.cognizant.spring_learn.Country">
    <property name="code" value="IN"/>
    <property name="name" value="India"/>
  </bean>
  <bean id="us" class="com.cognizant.spring_learn.Country">
    <property name="code" value="US"/>
    <property name="name" value="United States"/>
  </bean>
  <bean id="jp" class="com.cognizant.spring_learn.Country">
    <property name="code" value="JP"/>
    <property name="name" value="Japan"/>
  </bean>
  <bean id="de" class="com.cognizant.spring_learn.Country">
    <property name="code" value="DE"/>
    <property name="name" value="Germany"/>
  </bean>
  <bean id="countryList" class="java.util.ArrayList">
    <constructor-arg>
      <list>
        <ref bean="country"/>
        <ref bean="us"/>
        <ref bean="jp"/>
        <ref bean="de"/>
      </list>
    </constructor-arg>
  </bean>
</beans>
```

## CountryService.java

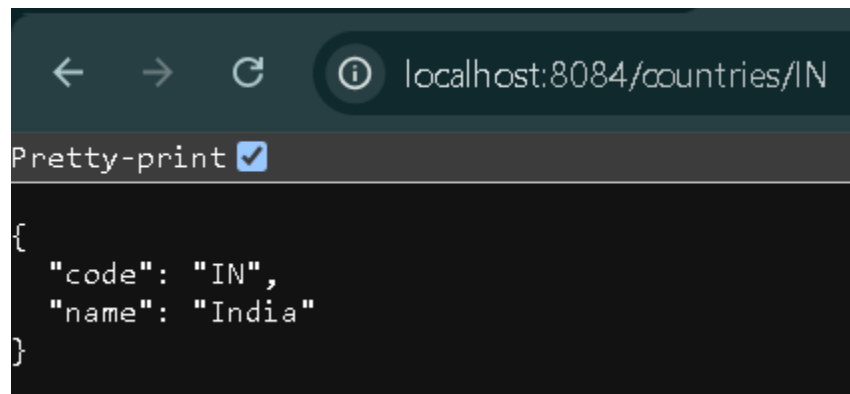
```
package com.cognizant.spring_learn.service;

import java.util.List;

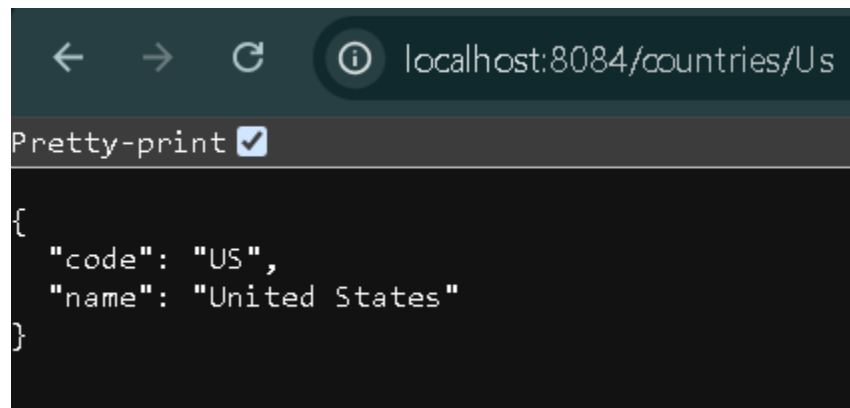
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Service;
import com.cognizant.spring_learn.Country;

@Service
public class CountryService {
    public Country getCountry(String code) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        List<Country> countryList =
            context.getBean("countryList", List.class);
        for (Country country : countryList) {
            if (country.getCode().equalsIgnoreCase(code)) {
                return country;
            }
        }
        return null;
    }
}
```

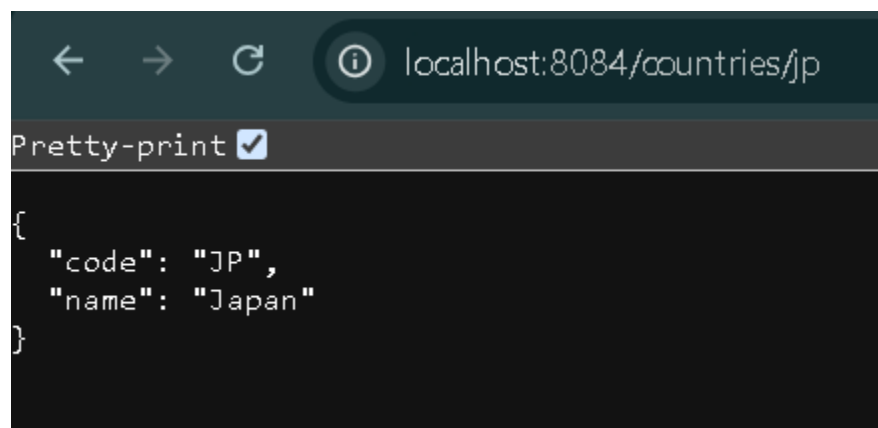
Outputs:



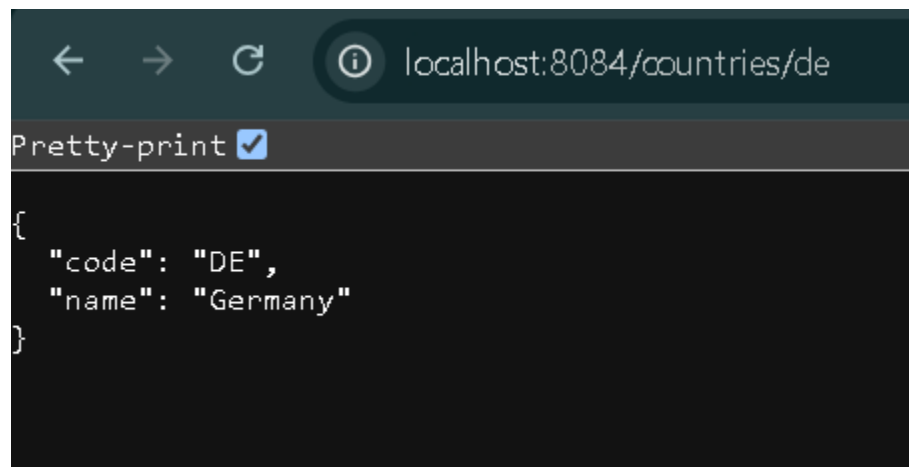
```
localhost:8084/countries/IN
Pretty-print ☒
{
  "code": "IN",
  "name": "India"
}
```



```
localhost:8084/countries/Us
Pretty-print ☒
{
  "code": "US",
  "name": "United States"
}
```



```
localhost:8084/countries/jp
Pretty-print ☒
{
  "code": "JP",
  "name": "Japan"
}
```



A screenshot of a web browser window. The address bar shows the URL `localhost:8084/countries/de`. Below the address bar, there is a toggle for "Pretty-print" which is checked. The main content area displays a JSON object: `{ "code": "DE", "name": "Germany" }`.

```
{  
  "code": "DE",  
  "name": "Germany"  
}
```

# JWT-HANDS ON

## Create authentication service that returns JWT

As part of first step of JWT process, the user credentials needs to be sent to authentication service request that generates and returns the JWT.

Ideally when the below curl command is executed that calls the new authentication service, the token should be responded. Kindly note that the credentials are passed using -u option.

### Request

```
curl -s -u user:pwd http://localhost:8090/authenticate
```

### Response

```
{"token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyIiwiaWF0IjoxNTcwMzc5NDc0LCJleHAiOjE1NzAzODA2NzR9.t3LRv1CV-hwKfoqZY1aVQqEUiB1oWcWn0ft3tgv0dL0"}
```

This can be incorporated as three major steps:

- Create authentication controller and configure it in SecurityConfig
- Read Authorization header and decode the username and password
- Generate token based on the user retrieved in the previous step

Let incorporate the above as separate hands on exercises.

Spring Security Dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

JWT Dependencies:

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.7</version>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.7</version>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.7</version>
  <scope>runtime</scope>
</dependency>
```

## SecurityConfig.java

```
package com.cognizant.spring_learn.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public InMemoryUserDetailsManager userDetailsService() {

        UserDetails admin = User.builder()
            .username("admin")
            .password(passwordEncoder().encode("pwd"))
            .roles("ADMIN")
            .build();
        UserDetails user = User.builder()
            .username("user")
            .password(passwordEncoder().encode("pwd"))
            .roles("USER")
            .build();

        return new InMemoryUserDetailsManager(admin, user);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .httpBasic(Customizer.withDefaults())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/countries").hasRole("USER")
                .anyRequest().authenticated());
        return http.build();
    }
}
```

AuthenticationResponse.java

```
package com.cognizant.spring_learn.model;

public class AuthenticationResponse {
    private String token;
    public AuthenticationResponse() {
    }
    public AuthenticationResponse(String token) {
        this.token = token;
    }
    public String getToken() {
        return token;
    }
    public void setToken(String token) {
        this.token = token;
    }
}
```

JwUtil.java

```
package com.cognizant.spring_learn.util;

import java.security.Key;
import java.util.Date;

import javax.crypto.SecretKey;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;

import org.springframework.stereotype.Component;

@Component
public class JwUtil {

    private static final SecretKey SECRET_KEY =
Keys.hmacShaKeyFor("ThisIsASecretKeyForJwtGeneration123456789".getBytes());

    public String generateToken(String username) {

        Date now = new Date();
        Date expiry = new Date(now.getTime() + 60 * 60 * 1000); // 1 hour
        return Jwts.builder()
            .subject(username)
            .issuedAt(now)
            .expiration(expiry)
            .signWith(SECRET_KEY)
            .compact();
    }
}
```

## AuthenticationController.java

```
package com.cognizant.spring_learn.controller;

import java.nio.charset.StandardCharsets;
import java.util.Base64;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpHeaders;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
import com.cognizant.spring_learn.model.AuthenticationResponse;
import com.cognizant.spring_learn.util.JwtUtil;
import jakarta.servlet.http.HttpServletRequest;

@RestController
public class AuthenticationController {
    @Autowired
    private JwtUtil jwtUtil;

    @GetMapping("/authenticate")
    public AuthenticationResponse authenticate(HttpServletRequest request) {
        String authorizationHeader =
            request.getHeader(HttpHeaders.AUTHORIZATION);
        String base64Credentials =
            authorizationHeader.substring("Basic ".length());
        byte[] decodedBytes =
            Base64.getDecoder().decode(base64Credentials);
        String credentials =
            new String(decodedBytes, StandardCharsets.UTF_8);
        String username =
            credentials.split(":")[0];
        String token = jwtUtil.generateToken(username);
        return new AuthenticationResponse(token);
    }
}
```

## OUTPUTS:

### 1. curl -v <http://localhost:8084/countries> [NORMAL LOGIN]

```
C:\Users\user>curl -v http://localhost:8084/countries* Host localhost:8084 was resolved.* IPv6: ::1* IPv4: 127.0.0.1* Trying [::1]:8084...* Established connection to localhost (::1 port 8084) from ::1 port 35385* using HTTP/1.x> GET /countries HTTP/1.1> Host: localhost:8084> User-Agent: curl/8.20.0> Accept: /*>* Request completely sent off< HTTP/1.1 401< Set-Cookie: JSESSIONID=5B004BF57E962567C0F1CBAFDD278252; Path=/; HttpOnly< WWW-Authenticate: Basic realm="Realm"< X-Content-Type-Options: nosniff< X-XSS-Protection: 0< Cache-Control: no-cache, no-store, max-age=0, must-revalidate< Pragma: no-cache< Expires: 0< X-Frame-Options: DENY< Content-Length: 0< Date: Thu, 02 Jul 2026 20:38:52 GMT<* Connection #0 to host localhost:8084 left intact
```

```
C:\Users\user>curl -v http://localhost:8084/countries
* Host localhost:8084 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8084...
* Established connection to localhost (::1 port 8084) from ::1 port 35385
* using HTTP/1.x
> GET /countries HTTP/1.1
> Host: localhost:8084
> User-Agent: curl/8.20.0
> Accept: /*
>
* Request completely sent off
< HTTP/1.1 401
< Set-Cookie: JSESSIONID=5B004BF57E962567C0F1CBAFDD278252; Path=/; HttpOnly
< WWW-Authenticate: Basic realm="Realm"
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< X-Frame-Options: DENY
< Content-Length: 0
< Date: Thu, 02 Jul 2026 20:38:52 GMT
<
* Connection #0 to host localhost:8084 left intact
```

### 2. curl -u user:pwd <http://localhost:8084/countries>

```
C:\Users\user>curl -u user:pwd http://localhost:8084/countries
```

```
[{"code":"IN","name":"India"}, {"code":"US","name":"United States"}, {"code":"JP","name":"Japan"}, {"code":"DE","name":"Germany"}]
```

### 3. curl -v -u user:pwd http://localhost:8084/authenticate

\* Host localhost:8084 was resolved.\* IPv6: ::1\* IPv4: 127.0.0.1\* Trying [::1]:8084...\* Established connection to localhost (::1 port 8084) from ::1 port 7748\* using HTTP/1.x\* Server auth using Basic with user 'user'> GET /authenticate HTTP/1.1> Host: localhost:8084> Authorization: Basic dXNlcjpwd2Q=> User-Agent: curl/8.20.0> Accept: \*/\*> Request completely sent off< HTTP/1.1 404< Vary: Origin< Vary: Access-Control-Request-Method< Vary: Access-Control-Request-Headers< X-Content-Type-Options: nosniff< X-XSS-Protection: 0< Cache-Control: no-cache, no-store, max-age=0, must-revalidate< Pragma: no-cache< Expires: 0< X-Frame-Options: DENY< Content-Type: application/json< Transfer-Encoding: chunked< Date: Thu, 02 Jul 2026 20:52:49 GMT<{"timestamp":"2026-07-02T20:52:49.769+00:00","status":404,"error":"Not Found"}

```
C:\Users\user>curl -v -u user:pwd http://localhost:8084/authenticate
* Host localhost:8084 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8084...
* Established connection to localhost (::1 port 8084) from ::1 port 7748
* using HTTP/1.x
* Server auth using Basic with user 'user'
> GET /authenticate HTTP/1.1
> Host: localhost:8084
> Authorization: Basic dXNlcjpwd2Q=
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 404
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< X-Frame-Options: DENY
< Content-Type: application/json
< Transfer-Encoding: chunked
< Date: Thu, 02 Jul 2026 20:52:49 GMT
<
{"timestamp":"2026-07-02T20:52:49.769+00:00","status":404,"error":"Not Found","trace":"org.springframework.web.servlet
```

#### 4. curl -v -u user:pwd <http://localhost:8084/authenticate>

C:\Users\user>curl -v -u user:pwd <http://localhost:8084/authenticate>

\* Host localhost:8084 was resolved.\* IPv6: ::1\* IPv4: 127.0.0.1\* Trying [::1]:8084...\* Established connection to localhost (::1 port 8084) from ::1 port 8989\* using HTTP/1.x\* Server auth using Basic with user 'user'> GET /authenticate HTTP/1.1> Host: localhost:8084> Authorization: Basic dXNlcjpwd2Q=> User-Agent: curl/8.20.0> Accept: \*/\*> Request completely sent off< HTTP/1.1 200< X-Content-Type-Options: nosniff< X-XSS-Protection: 0< Cache-Control: no-cache, no-store, max-age=0, must-revalidate< Pragma: no-cache< Expires: 0< X-Frame-Options: DENY< Content-Type: application/json< Transfer-Encoding: chunked< Date: Thu, 02 Jul 2026 21:05:54 GMT<{"token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2V5IiwiaWF0IjoxNzg5MDI2MzU0LCJleHAiOiJlE3ODMwMjk5NTR9.GUPxmTEXjHGg2yxldRdaq5-6lQXwemeW3EVnNuXyfKk"}\* Connection #0 to host localhost:8084 left intact

```
C:\Users\user>curl -v -u user:pwd http://localhost:8084/authenticate
* Host localhost:8084 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8084...
* Established connection to localhost (::1 port 8084) from ::1 port 12499
* using HTTP/1.x
* Server auth using Basic with user 'user'
> GET /authenticate HTTP/1.1
> Host: localhost:8084
> Authorization: Basic dXNlcjpwd2Q=
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200
< X-Content-Type-Options: nosniff
< X-XSS-Protection: 0
< Cache-Control: no-cache, no-store, max-age=0, must-revalidate
< Pragma: no-cache
< Expires: 0
< X-Frame-Options: DENY
< Content-Type: application/json
< Transfer-Encoding: chunked
< Date: Thu, 02 Jul 2026 21:20:37 GMT
<
{"token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2V5IiwiaWF0IjoxNzg5MDI3MjM3LCJleHAiOiJlE3ODMwMzA4Mzd9.LnWo2wbxB_RNruGzfGvJUKLTUm6NrC-xmb_tPt_N03k"}* Connection #0 to host localhost:8084 left intact
```

#### 5. Accessing Protected Endpoint Without Authentication (401 Unauthorized)

```
C:\Users\user>curl http://localhost:8084/countries
```

#### 6. Accessing Protected Endpoint with Invalid Credentials (401 Unauthorized)

```
C:\Users\user>curl -u user:wrongpwd http://localhost:8084/countries
```

#### 7. Generating JWT Token Using Authentication Endpoint

```
C:\Users\user>curl -u admin:pwd http://localhost:8084/countries
{"timestamp":"2026-07-02T21:20:13.218+00:00","status":403,"error":"Forbidden","message":"Forbidden","path":"/countries"}
```